

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:48:17

Conversation ID: 6a9bb0df-d570-83e8-9f49-34d8cc36ddcd

Conversation

User

```
namespace tlsfp { // Ground 1: Atomic handle for async-signal-safe termination static
std::atomic<pcap_t*> g_pcap_handle(nullptr); void signal_handler(int signum) { pcap_t *handle =
g_pcap_handle.load(std::memory_order_relaxed); if (handle != nullptr) { std::cout << "\n[*] Caught signal "
<< signum << ", stopping capture gracefully...\n"; pcap_breakloop(handle); } } void
CaptureContext::cleanup_stale_flows(time_t current_time) { constexpr time_t TIMEOUT_SECONDS = 30;
for (auto it = active_flows.begin(); it != active_flows.end(); ) { if (current_time - it->second.last_seen >
TIMEOUT_SECONDS) { it = active_flows.erase(it); } else { ++it; } } } Explain me these two functions in short
```

Assistant

Sure — in short: 1. Handles signals like Ctrl+C (). Gets the current capture handle from . If a valid handle exists, it calls . This tells the packet-capture loop to stop gracefully instead of abruptly terminating the program. So basically: Ctrl+C → signal handler → tell pcap to stop capturing. Small issue: is not async-signal-safe, so ideally the handler should avoid printing with . 2. Removes inactive/old network flows from . Goes through every active flow. Checks: If a flow hasn't been seen for more than 30 seconds, it is erased. Otherwise, it keeps the flow. So basically: remove flows that haven't had packets for 30 seconds.

Code Block 1 (text)

```
current_time - last_seen > 30
```

User

```
static std::atomic<pcap_t*> g_pcap_handle(nullptr); what is this bool start_capture(const CaptureOptions
&opts) { char errbuf[PCAP_ERRBUF_SIZE]; struct bpf_program fp; } and explain this also
```

Code Block 1 (text)

```
pcap_t *handle = nullptr;

if (!*opts.read_filename.empty()) {

    handle = pcap_open_offline(*opts.read_filename.c_str(), errbuf);
```

```

} else {

handle = pcap\_open\_live(*opts*.interface\_name.c\_str(), BUFSIZ, 1, 1000, errbuf);

}

if (!handle) {

std::cerr << "[-] Error opening pcap target: " << errbuf << "\n";

return false;

}

g\_pcap\_handle.store(handle, std::memory\_order\_relaxed);

```

Code Block 2 (text)

```

if (pcap\_compile(handle, &fp, "tcp port 443", 0, PCAP\_NETMASK\_UNKNOWN) == 0) {

pcap\_setfilter(handle, &fp);

pcap\_freecode(&fp);

} else {

std::cerr << "[-] Warning: Failed to set BPF filter 'tcp port 443'\n";

}

CaptureContext ctx;

ctx.link\_type = pcap\_datalink(handle);

std::cout << "[*] Capture initialized. Datalink type: " << ctx.link\_type << "\n";

if (!*opts*.write\_filename.empty()) {

ctx.dumper = pcap\_dump\_open(handle, *opts*.write\_filename.c\_str());

if (!ctx.dumper) {

std::cerr << "[-] Warning: Could not open output PCAP for writing: " << *opts*.write\_filename << "\n";

}

}

pcap\_loop(handle, 0, packet\_callback, reinterpret\_cast<u\_char*>(&ctx));

if (ctx.dumper) {

pcap\_dump\_flush(ctx.dumper);

```

```

pcap\_dump\_close(ctx.dumper);

}

pcap\_close(handle);

g\_pcap\_handle.store(nullptr, std::memory\_order\_relaxed);

std::cout << "[*] Capture completed.\n";

return true;

```

Assistant

Absolutely. Think of this code as setting up a packet capture, optionally filtering packets, processing them, and then cleaning everything up. 1. What is this? Break it down: `g_pcap_handle` is a pointer to a libpcap capture handle. It represents the active packet capture. `std::memory_order_relaxed` makes access to this pointer safe when it is shared with the signal handler. `g_pcap_handle` this variable belongs to this source file / namespace scope rather than being externally visible. `g_pcap_handle` initially, there is no active capture. So conceptually: The important reason it exists is your signal handler: When you press Ctrl+C, the handler needs to know which pcap capture to stop. This function starts and manages the entire packet-capture process. Step 1 – Open a capture source Initially, no capture exists. Then: There are two possibilities: PCAP file provided: No file provided: So your program can either analyze an existing capture or capture packets live. Step 2 – Check if opening worked If libpcap couldn't open the file/interface, print the error and stop. Step 3 – Make the handle available to the signal handler Now: So if the user presses Ctrl+C, can retrieve this handle and call: Step 4 – Set a BPF filter This filter means: Only capture TCP traffic on port 443. Port 443 is normally HTTPS/TLS traffic. Then: actually applies the filter. And: frees the compiled filter. Step 5 – Create the capture context This stores information needed while processing packets, such as: The datalink type tells you what kind of link-layer header the captured packets have, e.g. Ethernet. Step 6 – Optionally save packets to another PCAP If the user specified an output filename, packets can also be written into a new file. Step 7 – Start the actual capture loop This is the main part. repeatedly receives packets and calls: for each packet. The is passed to the callback so the callback can access your . Conceptually: If you press Ctrl+C: That's why is needed. Step 8 – Clean up After finishes: Flush and close the output PCAP if one was opened. Then: Close the capture itself. And importantly: means: There is no longer an active capture. Finally: indicates successful completion. Overall picture In one sentence: lets the signal handler know which capture to stop, while sets up the capture, filters TLS traffic, processes packets, and safely cleans everything up afterward.

Code Block 1 (text)

```

static std::atomic<pcap_t*> g_pcap_handle(nullptr);

```

Code Block 2 (text)

g_pcap_handle

⌘

nullptr ⌘ before capture

⌘

pcap_t* ⌘ while capture is running

⌘

nullptr ⌘ after capture finishes

Code Block 3 (text)

```
pcap_t *handle = g_pcap_handle.load(...);  
pcap_breakloop(handle);
```

Code Block 4 (text)

```
bool start_capture(const CaptureOptions &opts)
```

Code Block 5 (text)

```
pcap_t *handle = nullptr;
```

Code Block 6 (text)

```
if (!opts.read_filename.empty()) {  
    handle = pcap_open_offline(...);  
} else {  
    handle = pcap_open_live(...);  
}
```

Code Block 7 (text)

```
.pcap file ⌘ pcap_open_offline()
```

Code Block 8 (text)

```
Network interface ⌘ pcap_open_live()
```

Code Block 9 (text)

```
if (!handle) {  
    ...  
    return false;  
}
```

Code Block 10 (text)

```
g_pcap_handle.store(handle, std::memory_order_relaxed);
```

Code Block 11 (text)

```
g_pcap_handle  
⌵  
handle  
⌵  
active pcap capture
```

Code Block 12 (text)

```
pcap_breakloop(handle);
```

Code Block 13 (text)

```
pcap_compile(handle, &fp, "tcp port 443", ...);
```

Code Block 14 (text)

```
pcap_setfilter(handle, &fp);
```

Code Block 15 (text)

```
pcap_freecode(&fp);
```

Code Block 16 (text)

```
CaptureContext ctx;
```

Code Block 17 (text)

```
ctx.link_type = pcap_datalink(handle);
```

Code Block 18 (text)

```
if (!opts.write_filename.empty()) {  
    ctx.dumper = pcap_dump_open(...);  
}
```

Code Block 19 (text)

```
pcap_loop(handle, 0, packet_callback,  
reinterpret_cast<u_char*>(&ctx));
```

Code Block 20 (text)

```
packet_callback(...)
```

Code Block 21 (text)

```
pcap_loop()
❏
❏ ❏ packet ❏ packet_callback()
❏ ❏ packet ❏ packet_callback()
❏ ❏ packet ❏ packet_callback()
❏ ❏ packet ❏ packet_callback()
❏
❏ ❏ ❏ ...
```

Code Block 22 (text)

```
Ctrl+C
❏
signal_handler()
❏
pcap_breakloop()
❏
pcap_loop() stops
```

Code Block 23 (text)

```
pcap_dump_flush(ctx.dumper);
pcap_dump_close(ctx.dumper);
```

Code Block 24 (text)

```
pcap_close(handle);
```

Code Block 25 (text)

```
g_pcap_handle.store(nullptr, ...);
```

Code Block 26 (text)

```
return true;
```

Code Block 27 (text)

```
start_capture()
❏
❏
Open PCAP file / network interface
❏
❏
Store handle in g_pcap_handle
❏
❏
Apply "tcp port 443" filter
❏
❏
```

```
Create CaptureContext
```

```
{
```

```
{
```

```
pcap_loop()
```

```
{
```

```
    packet packet_callback()
```

```
    packet packet_callback()
```

```
    packet packet_callback()
```

```
}
```

```
Ctrl+C
```

```
{
```

```
signal_handler()
```

```
{
```

```
{
```

```
pcap_breakloop()
```

```
{
```

```
{
```

```
exit pcap_loop()
```

```
{
```

```
{
```

```
Close output file
```

```
{
```

```
{
```

```
pcap_close()
```

```
{
```

```
{
```

```
g_pcap_handle = nullptr
```

Assistant

Yes. This is the main packet-processing function. Its job is essentially: Take one captured packet & find TCP payload & reassemble it into a TLS record & identify ClientHello/ServerHello & calculate JA3/JA4 fingerprints. I'll go section by section, keeping it short. First: This is simply a character array used by libpcap to store an error message. For example, if: fails, libpcap puts something like: inside . So: prints the error. calls this function once for every captured packet. The three arguments are: & your & packet metadata, such as timestamp and captured length & actual raw packet bytes 1. Get your context Earlier you passed: to . Here you convert it back into: so you can access things like: 2. Periodically remove old flows is hexadecimal for 2047. This is a fast way of checking approximately: "Have we processed another 2048 packets?" Then: removes flows that haven't been seen for 30 seconds. 3. Figure out where the IP header starts The packet doesn't necessarily begin with the IP header. For Ethernet: So this: determines how many bytes to skip. For example: VLAN handling If Ethernet contains a VLAN tag: it skips the additional VLAN bytes. So ultimately: tells us: Where does the IP packet begin? 4. Determine IPv4 or IPv6 The first 4 bits of the IP header indicate: Then: handles IPv4. It checks: Meaning: Ignore UDP, ICMP, etc. We only want TCP. Then it extracts: For IPv6, it does essentially the same thing but also handles some IPv6 extension headers. Eventually it checks: Again: Only TCP packets are interesting. 5. Read the TCP header Now we know: It determines the TCP header size: Then extracts the ports: 6. Find the TCP payload

This calculates where TLS data starts: So: Then: Now points directly to the TLS data. 7. Find the flow A flow is basically identified by: If it's a new flow: the code checks: means TLS Handshake record. So this is a quick filter: "If this doesn't look like the beginning of a TLS handshake, ignore it." 8. TCP stream reassembly This is one of the most important parts. A TLS message can be split across multiple TCP packets: The code uses: and: to make sure packets arrive in the expected sequence. Then: appends the payload to the flow's buffer. So eventually: It also handles overlapping TCP segments and rejects unexpected gaps. 9. Validate TLS Once at least 5 bytes are available: it reads the TLS record header: Then: means Handshake. means TLS versions in the family. And: checks the maximum TLS record payload size. If invalid: the flow is discarded. 10. Identify ClientHello vs ServerHello This is the key part: TLS handshake types include: So: ClientHello It calls: Then calculates: So you get: It also extracts/prints the SNI. ServerHello It does the equivalent: then: giving: 11. Print the result For ClientHello, it prints something like: For ServerHello: 12. Save the packet If the user requested an output PCAP: the packet gets written to the output file. The entire function in one picture So if you need to explain this in your JA3/JA4 project presentation, the simplest description is: processes every captured packet, identifies TCP/TLS traffic, reassembles fragmented TLS records, detects ClientHello or ServerHello messages, parses them, and computes the corresponding JA3/JA3S and JA4/JA4S fingerprints.

Code Block 1 (text)

```
char errbuf[PCAP_ERRBUF_SIZE];
```

Code Block 2 (text)

```
pcap_open_live(...)
```

Code Block 3 (text)

```
"en0: No such device"
```

Code Block 4 (text)

```
std::cerr << errbuf;
```

Code Block 5 (text)

```
void packet_callback(  
    u_char *user_data,  
    const struct pcap_pkthdr *pkthdr,  
    const u_char *packet)
```

Code Block 6 (text)

```
CaptureContext *ctx =  
    reinterpret_cast<CaptureContext*>(user_data);
```

Code Block 7 (text)

```
reinterpret_cast<u_char*>(&ctx)
```

Code Block 8 (text)

```
CaptureContext*
```

Code Block 9 (text)

```
ctx->active_flows  
ctx->packet_counter  
ctx->dumper
```

Code Block 10 (text)

```
if ((++ctx->packet_counter & 0x7FF) == 0)
```

Code Block 11 (text)

```
ctx->cleanup_stale_flows(pkthdr->ts.tv_sec);
```

Code Block 12 (text)

```
Ethernet header  
⌘  
IP header  
⌘  
TCP header  
⌘  
TLS data
```

Code Block 13 (text)

```
switch (ctx->link_type)
```

Code Block 14 (text)

```
DLT_EN10MB ⌘ 14 bytes  
DLT_LINUX_SLL ⌘ 16 bytes  
DLT_RAW ⌘ 0 bytes
```

Code Block 15 (text)

```
if (ethertype == 0x8100)  
link_header_len += 4;
```

Code Block 16 (text)

```
link_header_len
```

Code Block 17 (text)

```
uint8_t ip_version =  
(packet[link_header_len] >> 4);
```

Code Block 18 (text)

```
4  IPv4  
6  IPv6
```

Code Block 19 (text)

```
if (ip_version == 4)
```

Code Block 20 (text)

```
if (ip_hdr->ip_p != IPPROTO_TCP)  
return;
```

Code Block 21 (text)

```
key.src_ip  
key.dst_ip
```

Code Block 22 (text)

```
if (next_proto != IPPROTO_TCP)  
return;
```

Code Block 23 (text)

```
const struct tcphdr *tcp_hdr =  
reinterpret_cast<const struct tcphdr*>(  
packet + link_header_len + ip_header_len);
```

Code Block 24 (text)

```
packet  
└─  
  └─ Ethernet  
    └─ IP  
      └─ TCP ─ tcp_hdr points here
```

Code Block 25 (text)

```
size_t tcp_header_len = tcp_hdr->th_off * 4;
```

Code Block 26 (text)

```
key.src_port = tcp_hdr->th_sport;  
key.dst_port = tcp_hdr->th_dport;
```

Code Block 27 (text)

```
size_t header_total_len =  
link_header_len +  
ip_header_len +  
tcp_header_len;
```

Code Block 28 (text)

```
Ethernet + IP + TCP  
⊠  
payload  
⊠  
TLS bytes
```

Code Block 29 (text)

```
const uint8_t *payload =  
packet + header_total_len;
```

Code Block 30 (text)

```
auto it = ctx->active_flows.find(key);
```

Code Block 31 (text)

```
Source IP  
Destination IP  
Source Port  
Destination Port  
IP version
```

Code Block 32 (text)

```
if (it == ctx->active_flows.end())
```

Code Block 33 (text)

```
if (payload[0] != 0x16)  
return;
```

Code Block 34 (text)

```
TCP packet 1: [TLS part 1]  
TCP packet 2: [TLS part 2]
```

TCP packet 3: [TLS part 3]

Code Block 35 (text)

```
buf.next_seq
```

Code Block 36 (text)

```
tcp_hdr->th_seq
```

Code Block 37 (text)

```
std::memcpy(buf.bytes + buf.len,  
payload,  
payload_len);
```

Code Block 38 (text)

```
packet 1 0 0  
packet 2 0 0 0 StreamBuffer  
packet 3 0 0  
0  
Complete TLS record
```

Code Block 39 (text)

```
if (buf.len < 5) return;
```

Code Block 40 (text)

```
Byte 0 0 Record type  
Byte 1 0 Version major  
Byte 2 0 Version minor  
Byte 3-4 0 Record length
```

Code Block 41 (text)

```
record_type == 0x16
```

Code Block 42 (text)

```
major_ver == 0x03
```

Code Block 43 (text)

```
tls_rec_len <= 16384
```

Code Block 44 (text)

```
ctx->active_flows.erase(key);  
return;
```

Code Block 45 (text)

```
uint8_t handshake_type = buf.bytes[5];
```

Code Block 46 (text)

```
0x01 ☐ ClientHello  
0x02 ☐ ServerHello
```

Code Block 47 (text)

```
if (handshake_type == 0x01)
```

Code Block 48 (text)

```
parse_client_hello(...)
```

Code Block 49 (text)

```
compute_ja3(...)  
compute_ja4(...)
```

Code Block 50 (text)

```
ClientHello  
☐  
Parse  
☐  
JA3  
JA4
```

Code Block 51 (text)

```
else if (handshake_type == 0x02)
```

Code Block 52 (text)

```
parse_server_hello(...)
```

Code Block 53 (text)

```
compute_ja3s(...)  
compute_ja4s(...)
```

Code Block 54 (text)

ServerHello

❏

Parse

❏

JA3S

JA4S

Code Block 55 (text)

[+] Captured ClientHello

❏ [SNI] example.com

❏ [JA3 String] ...

❏ [JA3 Hash] ...

❏ [JA4] ...

Code Block 56 (text)

[+] Captured ServerHello

❏ [JA3S String] ...

❏ [JA3S Hash] ...

❏ [JA4S] ...

Code Block 57 (text)

```
if (ctx->dumper) {  
    pcap_dump(...);  
}
```

Code Block 58 (text)

Captured packet

❏

Get CaptureContext

❏

Determine link-layer header

❏

Find IP header

❏

IPv4 / IPv6?

❏

Is it TCP?

❏

Find TCP payload

❏

Find/create flow

❏

Reassemble TCP segments

❏

Validate TLS record

❏

ClientHello or ServerHello?

```

//
// 1 2 3 4 5 6 7 8 9 10 11 12 13 14
//
// ClientHello  ServerHello
//
// Parse      Parse
//
// JA3 + JA4   JA3S + JA4S
//
// Print results + optionally dump packet

```

User

```

class LinkLayerDemuxTest : public ::testing::Test { protected: void SetUp() override { void
inject_packet(const std::vector<uint8_t>& pkt) { struct pcap_pkthdr hdr{}; }; TEST_F(LinkLayerDemuxTest,
HandlesDltRaw) {} TEST_F(LinkLayerDemuxTest, HandlesDltNull_Loopback) {}
TEST_F(LinkLayerDemuxTest, HandlesLinuxSLL) {} TEST_F(LinkLayerDemuxTest, RejectsUnknownDlt) {}
Tell me that what does each of the test check

```

Code Block 1 (text)

```

CaptureContext ctx;

```

Code Block 2 (text)

```

ctx.active\flows.clear();

}

std::vector<uint8_t> build_ip_tcp_payload(const std::vector<uint8_t>& *payload*) {

size_t ip_len = 20;

size_t tcp_len = 20;

std::vector<uint8_t> pkt(ip_len + tcp_len + *payload*.size(), 0);

pkt[0] = 0x45; // Version 4, IHL 5

uint16_t tot_len = htons(ip_len + tcp_len + *payload*.size());

std::memcpy(&pkt[2], &tot_len, 2);

pkt[9] = IPPROTO_TCP;

pkt[12] = 10; pkt[13] = 0; pkt[14] = 0; pkt[15] = 1;

pkt[16] = 10; pkt[17] = 0; pkt[18] = 0; pkt[19] = 2;

uint16_t sport = htons(12345), dport = htons(443);

```

```

uint32_t net_seq = htonl(1000); // Fixed SEQ for this test

std::memcpy(&pkt[20], &sport, 2);

std::memcpy(&pkt[22], &dport, 2);

std::memcpy(&pkt[24], &net_seq, 4);

pkt[32] = 0x50; // Data offset: 5 words

if (!*payload*.empty()) {

std::memcpy(&pkt[40], *payload*.data(), *payload*.size());

}

return pkt;

}

```

Code Block 3 (text)

```

hdr.caplen = *pkt*.size();

hdr.len = *pkt*.size();

hdr.ts.tv_sec = 1000;

packet_callback(reinterpret_cast<u_char*>(&ctx), &hdr, *pkt*.data());

}

```

Code Block 4 (text)

```

ctx.link_type = DLT_RAW;

std::vector<uint8_t> payload = {0x16, 0x03, 0x01, 0x00, 0x50};

std::vector<uint8_t> pkt = build_ip_tcp_payload(payload);

inject_packet(pkt);

EXPECT_EQ(ctx.active_flows.size(), 1);

```

Code Block 5 (text)

```

ctx.link_type = DLT_NULL;

std::vector<uint8_t> payload = {0x16, 0x03, 0x01, 0x00, 0x50};

std::vector<uint8_t> ip_pkt = build_ip_tcp_payload(payload);

```

```
std::vector<uint8_t> pkt(4 + ip_pkt.size(), 0);

pkt[0] = 0x02;

std::memcpy(&pkt[4], ip_pkt.data(), ip_pkt.size());

inject_packet(pkt);

EXPECT_EQ(ctx.active_flows.size(), 1);
```

Code Block 6 (text)

```
ctx.link_type = DLT_LINUX_SLL;

// LINUX_SLL (used when capturing on 'any') is 16 bytes.

// The last 2 bytes dictate the ethertype (e.g., 0x0800 for IPv4).

std::vector<uint8_t> payload = {0x16, 0x03, 0x01, 0x00, 0x50};

std::vector<uint8_t> ip_pkt = build_ip_tcp_payload(payload);

std::vector<uint8_t> pkt(16 + ip_pkt.size(), 0);

// Set Ethertype IPv4 (0x0800) at the end of the SLL header

pkt[14] = 0x08;

pkt[15] = 0x00;

std::memcpy(&pkt[16], ip_pkt.data(), ip_pkt.size());

inject_packet(pkt);

EXPECT_EQ(ctx.active_flows.size(), 1);
```

Code Block 7 (text)

```
// DLT_APPLE_IP_OVER_IEEE1394 or something unhandled

ctx.link_type = 138;

std::vector<uint8_t> payload = {0x16, 0x03, 0x01, 0x00, 0x50};

std::vector<uint8_t> pkt = build_ip_tcp_payload(payload);

inject_packet(pkt);

// Engine should gracefully ignore link types it cannot parse safely

EXPECT_EQ(ctx.active_flows.size(), 0);
```

Assistant

Yes. These are unit tests for the link-layer handling part of . They check whether your code correctly handles different DLT (Data Link Type) formats before reaching the IP/TCP/TLS data. First, what the test fixture does This creates a GoogleTest test fixture. Each test gets its own . clears before each test, so tests start clean. The helper: constructs a fake packet: And passes that fake packet directly to . 1. What it tests It checks that your callback correctly handles: For , there is no link-layer header, so: Therefore the packet starts directly with the IP header: The test injects a valid IPv4/TCP/TLS-looking packet and expects: Meaning The code should correctly parse a raw IP packet and create an active flow. 2. What it tests It checks: For , the link-layer header is 4 bytes: The test creates: and puts the IP packet starting at: Then it expects: Meaning The code correctly skips the 4-byte loopback header and parses the IP/TCP packet afterward. 3. What it tests It checks support for: which uses a 16-byte Linux cooked capture header. The test constructs: It also puts: which represents: Then the IP packet begins at: The expected result: Meaning The code correctly skips the 16-byte Linux SLL header and processes the IPv4/TCP packet. 4. This is the negative test. It deliberately gives the callback an unsupported DLT: Your callback has: So it should not try to interpret the packet. The test checks: Meaning If the link-layer type is unknown/unhandled, safely ignore the packet instead of attempting unsafe parsing. Easy way to remember the four TestDLTWhat it verifiesCorrectly handles 0-byte link headerCorrectly skips 4-byte loopback headerCorrectly skips 16-byte Linux SLL headerUnknown ()Rejects safely instead of parsing In one sentence for your project explanation: These tests verify that correctly calculates the IP-header offset for supported link-layer formats (RAW, NULL, and Linux SLL), while safely ignoring unsupported link-layer types.

Code Block 1 (text)

```
class LinkLayerDemuxTest : public ::testing::Test
```

Code Block 2 (text)

```
CaptureContext ctx;
```

Code Block 3 (text)

```
build_ip_tcp_payload(payload)
```

Code Block 4 (text)

```
IP header (20 bytes)
+
TCP header (20 bytes)
+
TLS payload
```

Code Block 5 (text)

```
TEST_F(LinkLayerDemuxTest, HandlesDltRaw)
```

Code Block 6 (text)

```
DLT_RAW
```

Code Block 7 (text)

```
link_header_len = 0;
```

Code Block 8 (text)

```
IP  TCP  TLS  
    
packet starts here
```

Code Block 9 (text)

```
EXPECT_EQ(ctx.active_flows.size(), 1);
```

Code Block 10 (text)

```
TEST_F(LinkLayerDemuxTest, HandlesDltNull_Loopback)
```

Code Block 11 (text)

```
DLT_NULL
```

Code Block 12 (text)

```
4-byte NULL header  
    
IP  
    
TCP  
    
TLS
```

Code Block 13 (text)

```
std::vector<uint8_t> pkt(4 + ip_pkt.size(), 0);
```

Code Block 14 (text)

```
&pkt[4]
```

Code Block 15 (text)

```
ctx.active_flows.size() == 1
```

Code Block 16 (text)

```
TEST_F(LinkLayerDemuxTest, HandlesLinuxSLL)
```

Code Block 17 (text)

```
DLT_LINUX_SLL
```

Code Block 18 (text)

16-byte SLL header

☒

IP

☒

TCP

☒

TLS

Code Block 19 (text)

```
pkt[14] = 0x08;  
pkt[15] = 0x00;
```

Code Block 20 (text)

```
0x0800 = IPv4
```

Code Block 21 (text)

```
pkt[16]
```

Code Block 22 (text)

```
ctx.active_flows.size() == 1
```

Code Block 23 (text)

```
TEST_F(LinkLayerDemuxTest, RejectsUnknownDlt)
```

Code Block 24 (text)

```
ctx.link_type = 138;
```

Code Block 25 (text)

```
default:
return;
```

Code Block 26 (text)

```
EXPECT_EQ(ctx.active_flows.size(), 0);
```

User

<https://github.com/salesforce/ja3/tree/master/python> see this repo, I am thinking about using some standard hashes from here to test my engine rather than AI guessing the hashes any good locations from where I can do this

Assistant

Yes — this is a much better approach than “AI guessing” the expected hashes. For your JA3/JA4 engine, you want known-good PCAPs + an independent implementation as the oracle. I checked the Salesforce repo. The important thing is that the Python implementation actually computes the JA3 string and MD5 digest from a captured ClientHello, and its README includes a concrete sample PCAP output with a known JA3 hash. GitHub+1 Best places to get test data 1. Salesforce JA3 repo — start here Salesforce JA3 Python implementation The README gives a sample result: So you can use their sample PCAP, run both: and assert: That's a legitimate correctness test rather than an AI-generated expected value. GitHub 2. Look for PCAPs in the actual Salesforce repo This is probably what I'd do next. The Python code accepts: and processes PCAP/PCAPNG files. GitHub You can therefore build a little test corpus: For each PCAP: That's far stronger than manually writing hashes into tests. 3. JA3er is useful for known fingerprints The Salesforce README specifically points to JA3er as a more up-to-date crowdsourced source of JA3 fingerprints. GitHub JA3er This is useful if you want tests such as: But there's an important distinction: A hash database alone isn't enough to test your parser. You really want: because the PCAP lets you verify that your engine actually parsed the TLS bytes correctly. 4. The Salesforce README itself has some useful canonical fingerprints It gives examples including: But don't use these alone as your primary tests. Salesforce explicitly describes them as examples, not fingerprints covering every version. GitHub What I'd recommend for your project Since your engine already does: I'd make three levels of tests. Level 1 — Unit tests The tests you just showed: These verify your packet parsing infrastructure. Level 2 — Golden PCAP tests ☒ This is what you're asking about. For example: Assert the entire fingerprint. Level 3 — Real-world applications Generate captures yourself: Capture their ClientHello traffic and run both implementations. This is especially valuable because your project's stated goal is distinguishing real clients/browsers/custom scripts by their TLS fingerprints. One important point about your current tests Your current test: is not a complete ClientHello. It's basically only enough to test: and your flow/reassembly logic. That's good for a unit test, but it cannot meaningfully test whether produces the correct fingerprint. For that, you want an actual captured ClientHello. Also, the Salesforce implementation's JA3

definition is specifically: followed by MD5 of that string. GitHub So I'd definitely use the Salesforce implementation as your reference/oracle, rather than having an LLM invent expected hashes. One caveat: the Salesforce repository was archived in May 2025, so for JA3 correctness/reference it's still useful, but you shouldn't treat it as the latest TLS fingerprinting standard. GitHub+1 If you want, I can also that would make a really solid golden-test suite for your C++ engine.

Code Block 1 (text)

```
JA3:
769,255-49162-49172-...-65279-10,0-10-11-35,23-24-25,0

MD5:
2aef69b4ba1938c3a400de4188743185
```

Code Block 2 (python)

```
Salesforce Python JA3
☐
expected JA3

Your C++ engine
☐
your JA3
```

Code Block 3 (text)

```
your JA3 == reference JA3
your MD5 == reference MD5
```

Code Block 4 (text)

```
ja3 --json your_file.pcap
```

Code Block 5 (text)

```
tests/
☐ ☐ pcaps/
☐ ☐ ☐ clienthello_1.pcap
☐ ☐ ☐ clienthello_2.pcap
☐ ☐ ☐ clienthello_3.pcap
☐ ☐ ☐ ...
☐ ☐ expected/
☐ ☐ fingerprints.json
```

Code Block 6 (text)

```
PCAP
☐
☐ ☐ Salesforce JA3 implementation ☐ expected
```

```

❏
❏ ❏ Your C++ implementation    ❏ actual
❏
compare
```

Code Block 7 (text)

```
curl    ❏ known JA3
Firefox ❏ known JA3
Chrome  ❏ known JA3
Python  ❏ known JA3
Tor     ❏ known JA3
```

Code Block 8 (text)

```
PCAP + expected fingerprint
```

Code Block 9 (text)

```
Tor      ❏ e7d705a3286e19ea42f587b344ee6865
Trickbot ❏ 6734f37431670b3ab4292b8f60f29984
Emotet   ❏ 4d7a28d6f2263ed61de88ca66eb011e3
```

Code Block 10 (text)

```
PCAP
❏
link layer
❏
IPv4/IPv6
❏
TCP
❏
TCP reassembly
❏
TLS validation
❏
ClientHello
❏
JA3 + JA4
```

Code Block 11 (text)

```
DLT_RAW
DLT_NULL
DLT_LINUX_SLL
Unknown DLT
```

Code Block 12 (text)

test_curl.pcap

☒

reference implementation

☒

JA3 = X

☒

your engine

☒

JA3 = X

Code Block 13 (text)

curl

Python requests

OpenSSL

Firefox

Chrome

Go TLS client

Code Block 14 (text)

```
std::vector<uint8_t> payload =  
{0x16, 0x03, 0x01, 0x00, 0x50};
```

Code Block 15 (text)

TLS record header

Code Block 16 (text)

SSLVersion,
CipherSuites,
Extensions,
EllipticCurves,
EllipticCurvePointFormats